

УТВЕРЖДАЮ

Руководитель разработки

_____ к.т.н., доц. Машкин М. Н.

«___» _____ 2026

**Программа визуализации
эквипотенциальных поверхностей
и силовых линий**

Вариант 5

Текст программы

Лист утверждения

643.02066606.0005-01 12 01-ЛУ

Инд. № подл.	Подпись и дата	Взам. инв. №	Инд. № дубл.	Подпись и дата

Исполнитель

Студент группы МЗО-225БВ-24

_____ Егоров А.В.

«___» _____ 2026

Москва 2026

УТВЕРЖДЕНО

643.02066606.0005-01 12 01-ЛУ

**Программа визуализации
эквипотенциальных поверхностей
и силовых линий**

Вариант 5

Текст программы

643.02066606.0005-01 12 01

Листов 21

Инов. № подл.	Подпись и дата	Взам. инв. №	Инов. № дубл.	Подпись и дата

1 ТЕКСТ ПРОГРАММЫ FIELDVIZ НА ИСХОДНОМ ЯЗЫКЕ

1.1 Исходный код файла main.cpp

Copyright. No Rights Reserved.
Егоров Александр Владиславович
E-mail: swrneko@mai.education
МАИ, Москва 2026

```
1  /*****
2  *                                  КАФЕДРА № 304 2 КУРС                                  *
3  *-----*
4  * Project Type   : Qt Widgets Desktop Application                                *
5  * Project Name   : FieldViz                                                       *
6  * File Name      : main.cpp                                                       *
7  * Language       : C/C++ (C++17)                                                 *
8  * Programmer(s)  : Егоров А.В. (swrneko)                                         *
9  * Modified By    : Егоров А.В. (swrneko)                                         *
10 * Edited by      : Neovim                                                         *
11 * OS             : Arch Linux                                                     *
12 * Created        : 01/07/26                                                       *
13 * Last Revision  : 01/07/26                                                       *
14 * Comment(s)     : Точка входа в программу.                                     *
15 *****/
16
17 #include <QApplication>
18 #include "MainWindow.h"
19
20 /*****
21 * Главная функция запуска программы      *
22 *****/
23 int main(int argc, char *argv[]) {
24     QApplication app(argc, argv); // Инициализация Qt-приложения
25
26     MainWindow window; // Создание главного окна
27     window.show(); // Отображение главного окна
28
29     return app.exec(); // Запуск цикла обработки событий Qt
30 }
31
```

1.2 Исходный код файла MainWindow.h

Copyright. No Rights Reserved.
 Егоров Александр Владиславович
 E-mail: swrneko@mai.education
 МАИ, Москва 2026

```

1  /*****
2  *                                     КАФЕДРА № 304 2 КУРС                                     *
3  *-----*
4  * Project Type   : Qt Widgets Desktop Application                                     *
5  * Project Name   : FieldViz                                                         *
6  * File Name      : MainWindow.h                                                     *
7  * Language       : C/C++ (C++17)                                                   *
8  * Programmer(s)  : Егоров А.В. (swrneko)                                           *
9  * Modified By    : Егоров А.В. (swrneko)                                           *
10 * Edited by      : Neovim                                                           *
11 * OS              : Arch Linux                                                      *
12 * Created        : 01/07/26                                                         *
13 * Last Revision  : 01/07/26                                                         *
14 * Comment(s)     : Главное окно GUI приложения.                                  *
15 *****/
16
17 #ifndef MAINWINDOW_H
18 #define MAINWINDOW_H
19
20 #include <QMainWindow>
21 #include "Visualizer.h"
22
23 class QCheckBox;
24 class QPushButton;
25 class QLabel;
26
27 /*****
28 * Класс главного окна приложения                                                     *
29 *****/
30 class MainWindow : public QMainWindow {
31     Q_OBJECT
32 public:
33     // Конструктор главного окна
34     explicit MainWindow(QWidget *parent = nullptr);
35
36 private slots:
37     // Слот переключения видимости силовых линий
38     void onFieldLinesToggled(bool checked);
39
40     // Слот переключения видимости эквипотенциалей

```

```
41     void onEquipotentialsToggled(bool checked);
42
43     // Слот сброса положения камеры
44     void onResetCameraClicked();
45
46 private:
47     Visualizer *m_visualizer;           // Виджет 3D-визуализации
48     QCheckBox *m_chkFieldLines;         // Чекбокс силовых линий
49     QCheckBox *m_chkEquipotentials;     // Чекбокс эквипотенциалей
50     QPushButton *m_btnResetCamera;     // Кнопка сброса камеры
51     QLabel *m_lblInfoPanel;            // Текстовая панель результатов (Solver)
52
53     // Инициализация интерфейса
54     void setupUi();
55 };
56
57 #endif // MAINWINDOW_H
58
```

1.3 Исходный код файла MainWindow.cpp

Copyright. No Rights Reserved.
 Егоров Александр Владиславович
 E-mail: swrneko@mai.education
 МАИ, Москва 2026

```

1  /*****
2  *                                КАФЕДРА № 304 2 КУРС                                *
3  *-----*
4  * Project Type   : Qt Widgets Desktop Application                                *
5  * Project Name   : FieldViz                                                    *
6  * File Name      : MainWindow.cpp                                              *
7  * Language       : C/C++ (C++17)                                             *
8  * Programmer(s)  : Егоров А.В. (swrneko)                                       *
9  * Modified By    : Егоров А.В. (swrneko)                                       *
10 * Edited by      : Neovim                                                      *
11 * OS              : Arch Linux                                                 *
12 * Created         : 01/07/26                                                  *
13 * Last Revision  : 01/07/26                                                  *
14 * Comment(s)     : Реализация логики главного окна приложения.              *
15 *****/
16
17 #include "MainWindow.h"
18 #include <QCheckBox>
19 #include <QPushButton>
20 #include <QLabel>
21 #include <QVBoxLayout>
22 #include <QHBoxLayout>
23 #include <QGroupBox>
24
25 /*****
26 * Конструктор главного окна                                *
27 *****/
28 MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent) {
29     setWindowTitle("КБН Портал: Визуализатор полей (Вариант 5)"); // Установка заголовка
30     resize(850, 600); // Базовый размер окна
31
32     setupUi(); // Инициализация интерфейса
33 }
34
35 /*****
36 * Инициализация элементов интерфейса                                *
37 *****/
38 void MainWindow::setupUi() {
39     QWidget *centralWidget = new QWidget(this); // Главный виджет
40     setCentralWidget(centralWidget); // Установка

```

```

41
42 QHBoxLayout *mainLayout = new QHBoxLayout(centralWidget); // Главный горизонтальный компоновщик
43
44 m_visualizer = new Visualizer(this); // Создание 3D-сцены
45 mainLayout->addWidget(m_visualizer, 3); // Добавление с весом 3
46
47 QVBoxLayout *sidebar = new QVBoxLayout(); // Вертикальный компоновщик сайдбара
48 mainLayout->addLayout(sidebar, 1); // Добавление с весом 1
49
50 // Секция 1: Управление видимостью
51 QGroupBox *grpControls = new QGroupBox("Отображение объектов", this); // Группа
52 QVBoxLayout *ctrlLayout = new QVBoxLayout(grpControls); // Компоновщик группы
53
54 m_chkFieldLines = new QCheckBox("Силовые линии поля a", this); // Чекбокс линий
55 m_chkFieldLines->setChecked(true); // Включен
56 connect(m_chkFieldLines, &QCheckBox::toggled, this, &MainWindow::onFieldLinesToggled); // Сигнал
57 ctrlLayout->addWidget(m_chkFieldLines); // Добавление
58
59 m_chkEquipotentials = new QCheckBox("Эквипотенциали поля b", this); // Чекбокс эквипотенциалей
60 m_chkEquipotentials->setChecked(true); // Включен
61 connect(m_chkEquipotentials, &QCheckBox::toggled, this, &MainWindow::onEquipotentialsToggled); // Сигнал
62 ctrlLayout->addWidget(m_chkEquipotentials); // Добавление
63
64 m_btnResetCamera = new QPushButton("Сбросить камеру", this); // Кнопка сброса
65 connect(m_btnResetCamera, &QPushButton::clicked, this, &MainWindow::onResetCameraClicked); // Сигнал
66 ctrlLayout->addWidget(m_btnResetCamera); // Добавление
67
68 sidebar->addWidget(grpControls); // Добавление группы в сайдбар
69
70 // Секция 2: Информационная панель (Solver)
71 QGroupBox *grpInfo = new QGroupBox("Аналитический расчёт", this); // Группа
72 QVBoxLayout *infoLayout = new QVBoxLayout(grpInfo); // Компоновщик
73
74 m_lblInfoPanel = new QLabel(this); // Текстовая метка
75 m_lblInfoPanel->setTextFormat(Qt::RichText); // Поддержка HTML
76 m_lblInfoPanel->setWordWrap(true); // Перенос строк
77 m_lblInfoPanel->setAlignment(Qt::AlignTop | Qt::AlignLeft); // Выравнивание
78
79 m_lblInfoPanel->setText(
80     "<b>Разработчик:</b><br>"
81     "Егоров А.В. (М30-225БВ-24)<br><br>"
82     "<b>Индивидуальный вариант 5:</b><br>"
83     "● Сфера S (1-й октант)<br><br>"
84     "<b>Поле a (Соленоидальное):</b><br>"
85     "<i>a = y2i + z2j + x2k</i><br>"
86     "● div a = 0, rot a ≠ 0<br>"
87     "● Поток P = 3π/16 ≈ 0.5890<br>"
88     "● Циркуляция C = -2<br><br>"

```

```

89     "<b>Поле b (Потенциальное):</b><br>"
90     "<i>b = (2xy3z+y)i + ...</i><br>"
91     "• rot b = 0<br>"
92     "• Потенциал U = xy + x2y3z + z<br>"
93     "• Работа W (0 → M) = 3"
94 ); // HTML Текст
95
96 infoLayout->addWidget(m_lblInfoPanel); // Добавление в группу
97 sidebar->addWidget(grpInfo); // Добавление группы в сайдбар
98
99 sidebar->addStretch(); // Растяжка для прижатия кверху
100 }
101
102 /*****
103  * Обработчики сигналов управления      *
104  *****/
105 void MainWindow::onFieldLinesToggled(bool checked) {
106     m_visualizer->setShowFieldLines(checked); // Переключение линий
107 }
108
109 void MainWindow::onEquipotentialsToggled(bool checked) {
110     m_visualizer->setShowEquipotentials(checked); // Переключение эквипотенциалей
111 }
112
113 void MainWindow::onResetCameraClicked() {
114     m_visualizer->resetCamera(); // Сброс камеры
115 }
116

```


1.4 Исходный код файла Visualizer.h

Copyright. No Rights Reserved.
 Егоров Александр Владиславович
 E-mail: swrneko@mai.education
 МАИ, Москва 2026

```

1  /*****
2  *                                     *
3  *-----*
4  * Project Type   : Qt Widgets Desktop Application *
5  * Project Name   : FieldViz *
6  * File Name      : Visualizer.h *
7  * Language       : C/C++ (C++17) *
8  * Programmer(s)  : Егоров А.В. (swrneko) *
9  * Modified By    : Егоров А.В. (swrneko) *
10 * Edited by      : Neovim *
11 * OS             : Arch Linux *
12 * Created        : 01/07/26 *
13 * Last Revision  : 01/07/26 *
14 * Comment(s)     : Виджет 3D визуализации полей. *
15 *****/
16
17 #ifndef VISUALIZER_H
18 #define VISUALIZER_H
19
20 #include <QWidget>
21 #include <QPoint>
22 #include <vector>
23 #include "FieldMath.h"
24
25 /*****
26 * Класс виджета визуализации полей и эквипотенциалей *
27 *****/
28 class Visualizer : public QWidget {
29     Q_OBJECT
30 public:
31     // Конструктор виджета
32     explicit Visualizer(QWidget *parent = nullptr);
33
34     // Установка отрисовки силовых линий
35     void setShowFieldLines(bool show);
36
37     // Установка отрисовки эквипотенциалей
38     void setShowEquipotentials(bool show);
39
40     // Сброс камеры в дефолтное положение

```

```
41     void resetCamera();
42
43 protected:
44     // Переопределение события рисования
45     void paintEvent(QPaintEvent *event) override;
46
47     // Переопределения мыши для вращения сцены
48     void mousePressEvent(QMouseEvent *event) override;
49     void mouseMoveEvent(QMouseEvent *event) override;
50     void wheelEvent(QWheelEvent *event) override;
51
52 private:
53     double m_yaw;    // Угол поворота вокруг оси Z (в градусах)
54     double m_pitch; // Угол поворота вокруг оси X (в градусах)
55     double m_zoom;   // Масштаб отображения
56     bool m_showFieldLines; // Флаг показа силовых линий
57     bool m_showEquipotentials; // Флаг показа эквипотенциалей
58
59     QPoint m_lastMousePos; // Последняя позиция курсора мыши
60
61     // Функция проекции 3D точки на 2D экран
62     QPointF project(const Point3D &p, int width, int height) const;
63
64     // Отрисовка координатных осей
65     void drawAxes(QPainter &painter, int w, int h);
66
67     // Отрисовка каркаса единичной сферы (1-й октант)
68     void drawSphereWireframe(QPainter &painter, int w, int h);
69
70     // Отрисовка силовых линий векторного поля a
71     void drawFieldLines(QPainter &painter, int w, int h);
72
73     // Отрисовка эквипотенциальных линий поля b
74     void drawEquipotentials(QPainter &painter, int w, int h);
75 };
76
77 #endif // VISUALIZER_H
78
```

1.5 Исходный код файла Visualizer.cpp

Copyright. No Rights Reserved.
Егоров Александр Владиславович
E-mail: swrneko@mai.education
МАИ, Москва 2026

```
1  /*****
2  *                                     КАФЕДРА № 304 2 КУРС *
3  *-----*
4  * Project Type   : Qt Widgets Desktop Application *
5  * Project Name   : FieldViz *
6  * File Name      : Visualizer.cpp *
7  * Language       : C/C++ (C++17) *
8  * Programmer(s)  : Егоров А.В. (swrneko) *
9  * Modified By    : Егоров А.В. (swrneko) *
10 * Edited by      : Neovim *
11 * OS             : Arch Linux *
12 * Created        : 01/07/26 *
13 * Last Revision  : 01/07/26 *
14 * Comment(s)     : Реализация функций 3D отрисовки сцены. *
15 *****/
16
17 #include "Visualizer.h"
18 #include <QPainter>
19 #include <QMouseEvent>
20 #include <QWheelEvent>
21 #include <cmath>
22 #include <algorithm>
23
24 /*****
25 * Конструктор и базовые настройки *
26 *****/
27 Visualizer::Visualizer(QWidget *parent)
28     : QWidget(parent), m_yaw(30.0), m_pitch(30.0), m_zoom(1.0),
29       m_showFieldLines(true), m_showEquipotentials(true) {
30     setMinimumSize(400, 400); // Минимальный размер виджета
31     setBackgroundRole(QPalette::Base); // Роль фона
32     setAutoFillBackground(true); // Заливка фона
33 }
34
35 /*****
36 * Управление флагами отрисовки *
37 *****/
38 void Visualizer::setShowFieldLines(bool show) {
39     m_showFieldLines = show; // Установка флага
40     update(); // Обновление окна
```

```
41  }
42
43  void Visualizer::setShowEquipotentials(bool show) {
44      m_showEquipotentials = show; // Установка флага
45      update(); // Обновление окна
46  }
47
48  void Visualizer::resetCamera() {
49      m_yaw = 30.0; // Сброс yaw
50      m_pitch = 30.0; // Сброс pitch
51      m_zoom = 1.0; // Сброс zoom
52      update(); // Обновление окна
53  }
54
55  /*****
56   * Проекция 3D точки на 2D экран
57   *****/
58  QPointF Visualizer::project(const Point3D &p, int width, int height) const {
59      double rad_yaw = m_yaw * M_PI / 180.0; // Перевод yaw в радианы
60      double rad_pitch = m_pitch * M_PI / 180.0; // Перевод pitch в радианы
61
62      // Поворот вокруг оси Z (Yaw)
63      double x1 = p.x * std::cos(rad_yaw) - p.y * std::sin(rad_yaw);
64      double y1 = p.x * std::sin(rad_yaw) + p.y * std::cos(rad_yaw);
65      double z1 = p.z;
66
67      // Поворот вокруг оси X (Pitch)
68      double x2 = x1;
69      double y2 = y1 * std::cos(rad_pitch) - z1 * std::sin(rad_pitch);
70
71      // Центрирование и масштаб
72      double scale = std::min(width, height) * 0.35 * m_zoom;
73      double sx = width / 2.0 + x2 * scale;
74      double sy = height / 2.0 - y2 * scale; // Инверсия Y экрана
75
76      return QPointF(sx, sy);
77  }
78
79  /*****
80   * Отрисовка координатных осей
81   *****/
82  void Visualizer::drawAxes(QPainter &painter, int w, int h) {
83      QPointF origin = project(Point3D(0, 0, 0), w, h); // Начало координат
84
85      // Ось X (Красная)
86      painter.setPen(QPen(Qt::red, 2)); // Перо
87      painter.drawLine(origin, project(Point3D(1.3, 0, 0), w, h)); // Линия
88      painter.drawText(project(Point3D(1.4, 0, 0), w, h), "X"); // Подпись
```

```

89
90 // Ось Y (Зелёная)
91 painter.setPen(QPen(Qt::darkGreen, 2)); // Перо
92 painter.drawLine(origin, project(Point3D(0, 1.3, 0), w, h)); // Линия
93 painter.drawText(project(Point3D(0, 1.4, 0), w, h), "Y"); // Подпись
94
95 // Ось Z (Синяя)
96 painter.setPen(QPen(Qt::blue, 2)); // Перо
97 painter.drawLine(origin, project(Point3D(0, 0, 1.3), w, h)); // Линия
98 painter.drawText(project(Point3D(0, 0, 1.4), w, h), "Z"); // Подпись
99 }
100
101 /*****
102 * Отрисовка каркаса сферы *
103 *****/
104 void Visualizer::drawSphereWireframe(QPainter &painter, int w, int h) {
105     painter.setPen(QPen(QColor(180, 180, 180, 150), 1, Qt::DashLine)); // Настройка пера
106
107     // Широты (параллели) сферы
108     for (int t_deg = 0; t_deg <= 90; t_deg += 15) {
109         double theta = t_deg * M_PI / 180.0; // В радианы
110         QPointF p_prev = project(Point3D(std::sin(theta), 0, std::cos(theta)), w, h); // Предыдущая
111         for (int p_deg = 3; p_deg <= 90; p_deg += 3) {
112             double phi = p_deg * M_PI / 180.0; // В радианы
113             Point3D pt(std::sin(theta) * std::cos(phi), std::sin(theta) * std::sin(phi), std::cos(theta));
114             QPointF p_curr = project(pt, w, h); // Спроецированная
115             painter.drawLine(p_prev, p_curr); // Рисование отрезка
116             p_prev = p_curr; // Сдвиг
117         }
118     }
119
120     // Долготы (меридианы) сферы
121     for (int p_deg = 0; p_deg <= 90; p_deg += 15) {
122         double phi = p_deg * M_PI / 180.0; // В радианы
123         QPointF p_prev = project(Point3D(0, 0, 1), w, h); // Старт на полюсе Z
124         for (int t_deg = 3; t_deg <= 90; t_deg += 3) {
125             double theta = t_deg * M_PI / 180.0; // В радианы
126             Point3D pt(std::sin(theta) * std::cos(phi), std::sin(theta) * std::sin(phi), std::cos(theta));
127             QPointF p_curr = project(pt, w, h); // Спроецированная
128             painter.drawLine(p_prev, p_curr); // Рисование отрезка
129             p_prev = p_curr; // Сдвиг
130         }
131     }
132 }
133
134 /*****
135 * Отрисовка силовых линий поля а *
136 *****/

```

```

137 void Visualizer::drawFieldLines(QPainter &painter, int w, int h) {
138     if (!m_showFieldLines) return; // Проверка флага
139
140     painter.setPen(QPen(Qt::darkCyan, 2)); // Перо для силовых линий
141
142     // Точки запуска линий (seeds)
143     std::vector<Point3D> seeds = {
144         Point3D(0.1, 0.1, 0.1), Point3D(0.3, 0.3, 0.3),
145         Point3D(0.5, 0.5, 0.5), Point3D(0.7, 0.7, 0.7),
146         Point3D(0.2, 0.5, 0.2), Point3D(0.5, 0.2, 0.5),
147         Point3D(0.1, 0.8, 0.1), Point3D(0.8, 0.1, 0.1)
148     };
149
150     for (const auto &seed : seeds) { // Проход по всем семенам
151         auto line = FieldMath::buildFieldLine(seed.x, seed.y, seed.z, 0.015, 120); // Расчёт линии
152         if (line.size() < 2) continue; // Пропуск коротких
153
154         QPointF p_prev = project(line[0], w, h); // Предыдущая
155         for (size_t i = 1; i < line.size(); ++i) { // Отрисовка по сегментам
156             QPointF p_curr = project(line[i], w, h); // Текущая
157             painter.drawLine(p_prev, p_curr); // Линия
158             p_prev = p_curr; // Сдвиг
159         }
160     }
161 }
162
163 /*****
164 * Отрисовка эквипотенциалей поля b      *
165 *****/
166 void Visualizer::drawEquipotentials(QPainter &painter, int w, int h) {
167     if (!m_showEquipotentials) return; // Проверка флага
168
169     painter.setPen(QPen(Qt::magenta, 1, Qt::SolidLine)); // Розовый цвет для U = const
170
171     // Константные уровни потенциала C для отрисовки
172     std::vector<double> levels = {0.1, 0.3, 0.5, 0.7, 0.9, 1.1, 1.3, 1.5};
173
174     // Отрисовка эквипотенциалей в плоскости xy (z = 0 -> U = xy = C)
175     for (double C : levels) {
176         QPointF p_prev; // Точка старта
177         bool first = true; // Флаг первой точки
178         for (double x = C; x <= 1.0; x += 0.02) { // Построение гиперболы y = C / x
179             double y = C / x; // Вычисление Y
180             if (y > 1.0) continue; // Ограничение
181             QPointF p_curr = project(Point3D(x, y, 0), w, h); // Проекция
182             if (!first) painter.drawLine(p_prev, p_curr); // Отрисовка
183             p_prev = p_curr; // Сдвиг
184             first = false; // Сброс флага

```

```

185     }
186 }
187
188 // Отрисовка на самой сфере методом сканирования сетки
189 // Если изоуровень C находится между вершинами ячейки сферы, рисуем отрезок
190 for (double C : levels) {
191     for (int t_deg = 0; t_deg < 90; t_deg += 3) {
192         double theta = t_deg * M_PI / 180.0;
193         double theta_next = (t_deg + 3) * M_PI / 180.0;
194         for (int p_deg = 0; p_deg < 90; p_deg += 3) {
195             double phi = p_deg * M_PI / 180.0;
196             double phi_next = (p_deg + 3) * M_PI / 180.0;
197
198             // Координаты 4-х вершин ячейки сферы
199             Point3D v00(std::sin(theta)*std::cos(phi), std::sin(theta)*std::sin(phi), std::cos(theta));
200             Point3D v10(std::sin(theta_next)*std::cos(phi), std::sin(theta_next)*std::sin(phi), std::cos(theta_next));
201             Point3D v01(std::sin(theta)*std::cos(phi_next), std::sin(theta)*std::sin(phi_next), std::cos(theta));
202
203             // Потенциалы в вершинах
204             double u00 = FieldMath::getPotentialU(v00.x, v00.y, v00.z);
205             double u10 = FieldMath::getPotentialU(v10.x, v10.y, v10.z);
206             double u01 = FieldMath::getPotentialU(v01.x, v01.y, v01.z);
207
208             // Линейная интерполяция изолинии C
209             if ((u00 <= C && u10 >= C) || (u00 >= C && u10 <= C)) {
210                 double t = (C - u00) / (u10 - u00);
211                 Point3D pt(v00.x + t*(v10.x - v00.x), v00.y + t*(v10.y - v00.y), v00.z + t*(v10.z - v00.z));
212                 painter.drawPoint(project(pt, w, h));
213             }
214             if ((u00 <= C && u01 >= C) || (u00 >= C && u01 <= C)) {
215                 double t = (C - u00) / (u01 - u00);
216                 Point3D pt(v00.x + t*(v01.x - v00.x), v00.y + t*(v01.y - v00.y), v00.z + t*(v01.z - v00.z));
217                 painter.drawPoint(project(pt, w, h));
218             }
219         }
220     }
221 }
222 }
223
224 /*****
225 * Обработчик события рисования виджета *
226 *****/
227 void Visualizer::paintEvent(QPaintEvent *event) {
228     Q_UNUSED(event); // Избегаем предупреждения компилятора
229     QPainter painter(this); // Инициализация рисовальщика
230     painter.setRenderHint(QPainter::Antialiasing); // Сглаживание линий
231
232     int w = width(); // Ширина окна

```

```
233     int h = height(); // Высота окна
234
235     // Очистка фона (белый цвет)
236     painter.fillRect(rect(), Qt::white);
237
238     drawAxes(painter, w, h); // 1. Отрисовка осей координат
239     drawSphereWireframe(painter, w, h); // 2. Отрисовка сетки сферы
240     drawFieldLines(painter, w, h); // 3. Отрисовка силовых линий
241     drawEquipotentials(painter, w, h); // 4. Отрисовка эквипотенциалей
242 }
243
244 /*****
245  * Обработка событий мыши
246  *****/
247 void Visualizer::mousePressEvent(QMouseEvent *event) {
248     if (event->button() == Qt::LeftButton) { // Проверка ЛКМ
249         m_lastMousePos = event->pos(); // Сохранение позиции клика
250     }
251 }
252
253 void Visualizer::mouseMoveEvent(QMouseEvent *event) {
254     if (event->buttons() & Qt::LeftButton) { // Перетаскивание ЛКМ
255         int dx = event->x() - m_lastMousePos.x(); // Смещение по X
256         int dy = event->y() - m_lastMousePos.y(); // Смещение по Y
257
258         m_yaw += dx * 0.5; // Изменение yaw
259         m_pitch += dy * 0.5; // Изменение pitch
260
261         m_lastMousePos = event->pos(); // Обновление позиции
262         update(); // Перерисовка окна
263     }
264 }
265
266 void Visualizer::wheelEvent(QWheelEvent *event) {
267     double numDegrees = event->angleDelta().y() / 8.0; // Угол поворота колеса
268     double numSteps = numDegrees / 15.0; // Число шагов
269
270     m_zoom += numSteps * 0.1; // Изменение зума
271     m_zoom = std::clamp(m_zoom, 0.2, 5.0); // Ограничение диапазона
272
273     update(); // Перерисовка окна
274 }
275
```


1.6 Исходный код файла FieldMath.h

Copyright. No Rights Reserved.
Егоров Александр Владиславович
E-mail: swrneko@mai.education
МАИ, Москва 2026

```
1  /*****
2  *                                     КАФЕДРА № 304 2 КУРС                                     *
3  *-----*
4  * Project Type   : Qt Widgets Desktop Application                                     *
5  * Project Name   : FieldViz                                                         *
6  * File Name      : FieldMath.h                                                       *
7  * Language       : C/C++ (C++17)                                                    *
8  * Programmer(s)  : Егоров А.В. (swrneko)                                             *
9  * Modified By    : Егоров А.В. (swrneko)                                             *
10 * Edited by      : Neovim                                                            *
11 * OS             : Arch Linux                                                         *
12 * Created        : 01/07/26                                                           *
13 * Last Revision  : 01/07/26                                                           *
14 * Comment(s)     : Математические функции расчёта полей.                           *
15 *****/
16
17 #ifndef FIELDMATH_H
18 #define FIELDMATH_H
19
20 #include <vector>
21
22 /*****
23 * Структура для представления точки в 3D *
24 *****/
25 struct Point3D {
26     double x; // Координата X
27     double y; // Координата Y
28     double z; // Координата Z
29
30     // Конструктор по умолчанию
31     Point3D() : x(0), y(0), z(0) {}
32
33     // Конструктор инициализации
34     Point3D(double px, double py, double pz) : x(px), y(py), z(pz) {}
35 };
36
37 /*****
38 * Класс математических расчётов полей*
39 *****/
40 class FieldMath {
```

```
41 public:
42     // Расчёт векторного поля a
43     static void getFieldA(double x, double y, double z, double &ax, double &ay, double &az);
44
45     // Расчёт векторного поля b
46     static void getFieldB(double x, double y, double z, double &bx, double &by, double &bz);
47
48     // Расчёт потенциала поля b
49     static double getPotentialU(double x, double y, double z);
50
51     // Численное построение силовой линии поля a (метод Рунге-Кутты 4 порядка)
52     static std::vector<Point3D> buildFieldLine(double sx, double sy, double sz, double step, int max_s
53 };
54
55 #endif // FIELDMATH_H
56
```

1.7 Исходный код файла FieldMath.cpp

Copyright. No Rights Reserved.
Егоров Александр Владиславович
E-mail: swrneko@mai.education
МАИ, Москва 2026

```
1  /*****
2  *                                     КАФЕДРА № 304 2 КУРС *
3  *-----*
4  * Project Type   : Qt Widgets Desktop Application *
5  * Project Name   : FieldViz *
6  * File Name      : FieldMath.cpp *
7  * Language       : C/C++ (C++17) *
8  * Programmer(s)  : Егоров А.В. (swrneko) *
9  * Modified By    : Егоров А.В. (swrneko) *
10 * Edited by      : Neovim *
11 * OS             : Arch Linux *
12 * Created        : 01/07/26 *
13 * Last Revision  : 01/07/26 *
14 * Comment(s)     : Реализация расчёта векторных полей. *
15 *****/
16
17 #include "FieldMath.h"
18 #include <cmath>
19
20 /*****
21 * Вспомогательная функция нормализации *
22 *****/
23 static Point3D getDirA(double x, double y, double z) {
24     double ax, ay, az; // Векторы поля
25     FieldMath::getFieldA(x, y, z, ax, ay, az); // Получение поля
26
27     double len = std::sqrt(ax * ax + ay * ay + az * az); // Длина вектора
28     if (len < 1e-9) { // Проверка на ноль
29         return Point3D(0, 0, 0); // Нулевой вектор
30     }
31
32     return Point3D(ax / len, ay / len, az / len); // Возврат нормали
33 }
34
35 /*****
36 * Расчёт векторного поля а (Вариант 5) *
37 *****/
38 void FieldMath::getFieldA(double x, double y, double z, double &ax, double &ay, double &az) {
39     ax = y * y; // a_x = y^2
40     ay = z * z; // a_y = z^2
```

```

41     az = x * x; // a_z = x^2
42 }
43
44 /*****
45 * Расчёт векторного поля b (Вариант 5) *
46 *****/
47 void FieldMath::getFieldB(double x, double y, double z, double &bx, double &by, double &bz) {
48     bx = 2.0 * x * y * y * y * z + y; // b_x = 2xy^3z + y
49     by = 3.0 * x * x * y * y * z + x; // b_y = 3x^2y^2z + x
50     bz = x * x * y * y * y + 4.0 * z * z * z; // b_z = x^2y^3 + 4z^3
51 }
52
53 /*****
54 * Расчёт потенциала поля b (Вариант 5) *
55 *****/
56 double FieldMath::getPotentialU(double x, double y, double z) {
57     return x * y + x * x * y * y * y * z + z * z * z * z; // U = xy + x^2y^3z + z^4
58 }
59
60 /*****
61 * Численное интегрирование силовой линии (метод Рунге-Кутты 4) *
62 *****/
63 std::vector<Point3D> FieldMath::buildFieldLine(double sx, double sy, double sz, double step, int max_steps) {
64     std::vector<Point3D> line; // Хранилище точек линии
65     Point3D curr(sx, sy, sz); // Начальная точка
66
67     line.push_back(curr); // Запись старта
68
69     for (int i = 0; i < max_steps; ++i) { // Цикл шагов
70         Point3D k1 = getDirA(curr.x, curr.y, curr.z); // Шаг 1
71         if (k1.x == 0 && k1.y == 0 && k1.z == 0) break; // Останов на нуле
72
73         Point3D p2(curr.x + 0.5 * step * k1.x, curr.y + 0.5 * step * k1.y, curr.z + 0.5 * step * k1.z);
74         Point3D k2 = getDirA(p2.x, p2.y, p2.z); // Шаг 2
75
76         Point3D p3(curr.x + 0.5 * step * k2.x, curr.y + 0.5 * step * k2.y, curr.z + 0.5 * step * k2.z);
77         Point3D k3 = getDirA(p3.x, p3.y, p3.z); // Шаг 3
78
79         Point3D p4(curr.x + step * k3.x, curr.y + step * k3.y, curr.z + step * k3.z); // Точка 4
80         Point3D k4 = getDirA(p4.x, p4.y, p4.z); // Шаг 4
81
82         curr.x += (step / 6.0) * (k1.x + 2.0 * k2.x + 2.0 * k3.x + k4.x); // Новый X
83         curr.y += (step / 6.0) * (k1.y + 2.0 * k2.y + 2.0 * k3.y + k4.y); // Новый Y
84         curr.z += (step / 6.0) * (k1.z + 2.0 * k2.z + 2.0 * k3.z + k4.z); // Новый Z
85
86         // Проверка выхода из первого октанта единичной сферы
87         double dist2 = curr.x * curr.x + curr.y * curr.y + curr.z * curr.z; // Квадрат расстояния
88         if (dist2 > 1.2 || curr.x < -0.1 || curr.y < -0.1 || curr.z < -0.1) { // Условие выхода

```

```
89         break; // Выход из цикла
90     }
91
92     line.push_back(curr); // Запись точки
93 }
94
95 return line; // Возврат линии
96 }
97
```

[illegible]